Module-XI

Lecture-III and IV

Memory Testing

1. **Introduction: Why is memory testing different?**

   Till now we have been looking into VLSI testing, only from the context where the circuit is composed of logic gates and flip-flops. However, memory blocks form a very important part of digital circuits but are not composed of logic gates and flip-flops. This necessitates different fault models and test techniques for memory blocks.

   In memory technology, the capacity quadruples roughly every 3 years [1], which leads to decrease in memory price per bit (being stored). It is to be noted that increase in capacity does not lead to a proportional rise in area of a memory chip. In other words, high storage capacity is obtained by raise in density, which implies decrease in the size of circuit (capacitor) used to store a bit. Experiments with new materials having high dielectric constant like barium strontium titanate [2] are being done that facilitate greater capacitance to be maintained in the same physical space. Also, design and layout techniques like stacked capacitors are being tried to achieve higher density. Further, for faster access of the memory, various methods are being developed which includes fast page mode (FP), extended data output (EDO), synchronous DRAM (SDRAM), double data rate etc.

   So, modern memory designs aim at high capacity at lower area and fast access speed. This leads to a situation where we have very less charge stored per memory cell (because of low capacitance), and cells are extremely close to each other. Further, due to use of deep micron technology the cells are becoming more susceptible to manufacturing defects.

   - *So, unlike normal circuits where we generally do not have faults (or chips having faults are discarded), multiple faults will be present in any memory chip. In other words, the yield of memory chips would be nearly 0%, since every chip has defects. During manufacturing test, the faults are not only to be detected but also*

their locations (in terms of cell number) are to be diagnosed. As almost all memories will have faults in some cells, there are redundant (extra) cells in the memory. One a fault is diagnosed, the corresponding cell is disconnected and a new fault free cell is connected in the appropriate position. This replacement is achieved by blowing fuses (using laser) to reroute defective cells to normal spare cells.

- *The sole functionality of a cell is to sore a bit information which is implemented using a capacitor; when the capacitor is charged it represents 1 and when there is no charge it represents 0. No logic gates are involved in a memory. Use of logic gates (in flip-flops) instead of capacitors to store bit information would lead to a very large area.*

The above two points basically differentiate testing of logic gate circuits from memory.

So, new fault models and test procedures are required for testing memories. In this lecture we will study the most widely used fault models and test techniques for such fault models in memories.

2. **Memory fault models**

As in the case of gate level digital circuits, testing the physical defects of memory is not possible. The other possible testing mechanism is to see if each cell can be written with the desired value (0/1) and be read back, which necessitates modeling of physical defects as logical faults. In case of memory, if logical fault model is to be applied then first a logic functional model is to be developed. Modeling faults as logical faults in a logical functional model makes the testing approach independent of the technology and the manufacturing process. Figure 1 shows logic functional model of a memory.

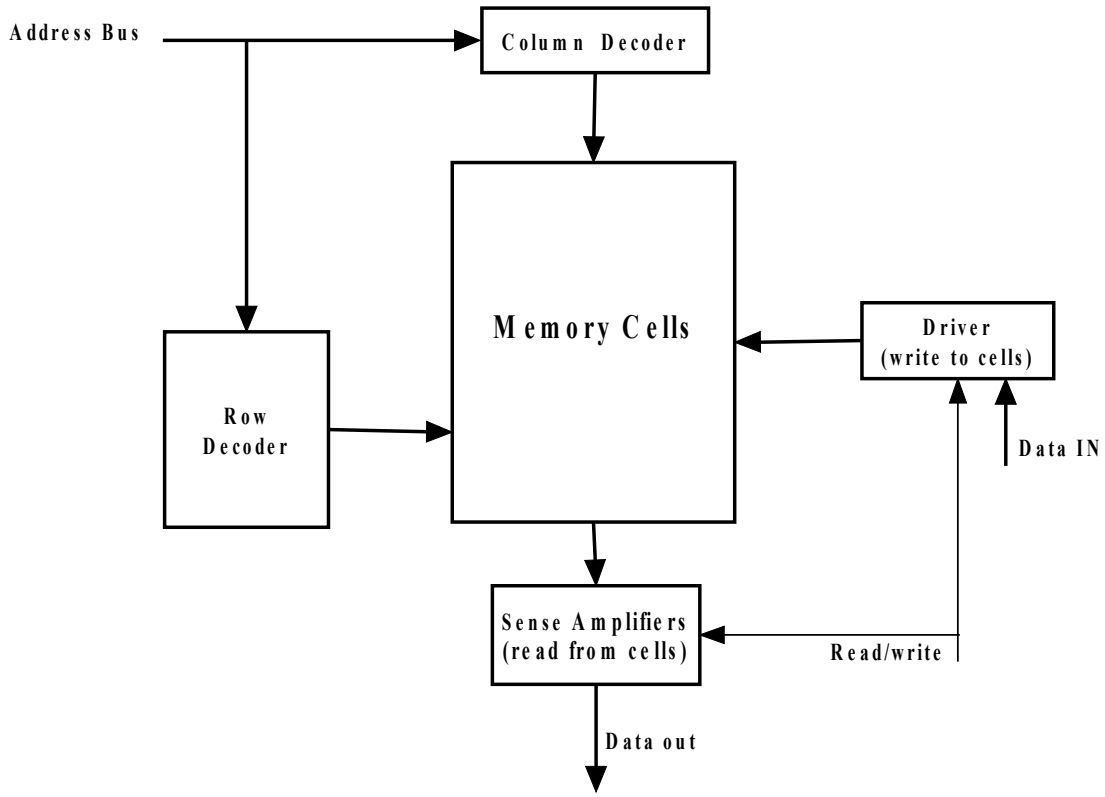


Figure 1. Logic functional model of memory

When data is to be read from the memory, first the row and column decoders determine the location (i.e., the cell) from the address (sent in the address bus) that needs to be accessed. Based on the address in the row and column decoders the cell of the appropriate row and column gets connected to the sense amplifier, which sends the data out. Similar situation (for accessing the required cells) holds when data is to be written in the memory, however, in case of writing, special driver circuitry writes the values in the cells from the data bus. Also there is a read/write select line in the memory. This is a very

broad level view of how memory works and detailed information can be found in [3]. For testing of memory this top level idea of its working suffices. It may be noted that from the testing perspective we would only check if

- Required value (0/1) can be written to a cell
- The stored value can be read from a cell
- The proper cell is accessed, i.e., the row and column decoder do not have faults.

It may be noted that the row and column decoders are digital circuits implemented using logic gates (which are different from memory cell implementation). The sense amplifier and driver are analog circuits. In testing of memory, we do not consider the decoders as gate level digital circuits nor the sense amplifier and driver as analog circuits. For the decoders, we test the functionality whether they can access the desired cells based on the address in the address bus. For the amplifier and driver we check if they can pass the values to and from the cells correctly.

There can be a large number of logical faults based on the memory model of Figure 1. However, the following faults called "reduced functional faults" are sufficient for functional memory testing [4,5].

- Stuck-at fault
- Transition fault
- Coupling fault
- Neighborhood pattern sensitive fault
- Address decoder faults

In this section we will elaborate on these fault models. For detailed discussion on exhaustive list of faults and modeling techniques the student is referred to [4,5,6].

2.1 **Stuck-at fault**

Stuck-at-fault in memory is the one in which the logic value of a cell (or line in the sense amplifier or driver) is always 0 or always 1.

Figure 2 shows the state transition diagram for a good memory cell. S0 is the state where the cell contains 0, while is S1 is the state where it contains 1. w1 (w0) indicates value of (0) 1 being written. The self loop from state S0 indicates 0 being written to the cell. Similarly, all transitions can be explained.

Figure 3 illustrates state diagrams for a cell under s-a-0 and s-a-1 condition.

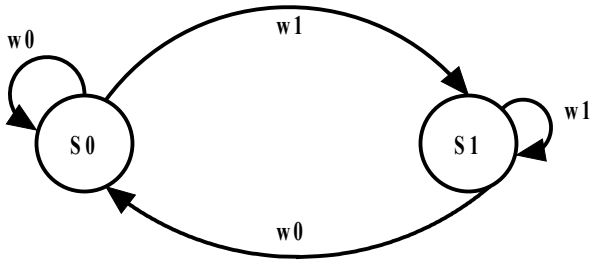


Figure 2. State diagram for a good memory cell

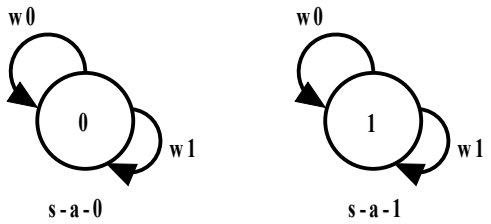


Figure 3. State diagram for a s-a1 memory cell and s-a-0 memory cell

2.2 **Transition fault**

In transition fault a cell fails to make a (0 to 1) transition or a (1 to 0) transition when it is written; up transition fault is denoted as $\langle \uparrow | 0 \rangle$ and a down transition fault is denoted as $\langle \downarrow | 1 \rangle$. Figure 4 shows state diagram for up transition fault. It may be noted from Figure 4 that a cell having up transition fault is same a s-a-0 fault, however, the cell can take and retain value 1 if a 0 has not yet been written to the cell. The dual happens for down transition fault.



Figure 4. State diagram for up transition fault in a memory cell

2.3 **Coupling Faults:**

Coupling fault, as the name suggests, implies deviation from normal behavior of a cell because of coupling with others. As there can be exponential number of combinations of coupling of a cell with others cells, we assume that in coupling faults a faulty cell can get coupled with another faulty cell. In other words, in the widely used coupling fault model it is assumed that any "two" cells can couple and normal behavior changes in these two cells; it is called 2-coupling fault model. So if there are n cells in a memory then there can be ${}^{n}C_{2}$ number of 2-coupling faults. To reduce the number of 2-coupling faults further from ${}^{n}C_{2}$, we assume that only neighboring cells (decided on threshold distance) can be involved in the fault. We consider two types of coupling faults namely, (i) inversion coupling faults and (ii) idempotent coupling faults. Now we elaborate them as follows.

2.3.1 Inversion coupling faults.

In a 2-inversion coupling fault $cfinv_{i,j}$ say, involving cells i and j, a transition (0 to1 or 1 to 0) in memory cell j causes an unwanted change in memory cell i. Memory cell i is the *coupled* cell (where fault occurs) and memory cell j is the *coupling* cell. The two possible 2-inversion coupling faults involving cells i, j (denoted as $cfinv_{i,j}$) are

- Rising: $\langle \uparrow | \updownarrow \rangle$ (implying 0 to1 change in cell j complements the content of cell i)
- Falling: $\langle \downarrow | \updownarrow \rangle$ (implying 1 to 0 change in cell j complements the content of cell i)

Figure 5 illustrates the state diagram for two cells i and j under normal condition. State $S00$ implies that both the cells have 0 values; state $S01$ implies that cell i contains 0 and cell j contains 1. Similarly all the four states can be explained. The self loop at state $S00$, marked w0@i implies that if 0 is written to cell i then the same state is retained; another transition w0@j is associated with the same self loop which implies that if 0 is written to cell j then $S00$ retained. However, if we write 1 to cell j (from state $S00$

) i.e., $w1@j$, we go to state $S01$; this is indicated by the transition from $S00$ to $S01$ marked $w1@j$. Similarly the whole state machine can be explained.

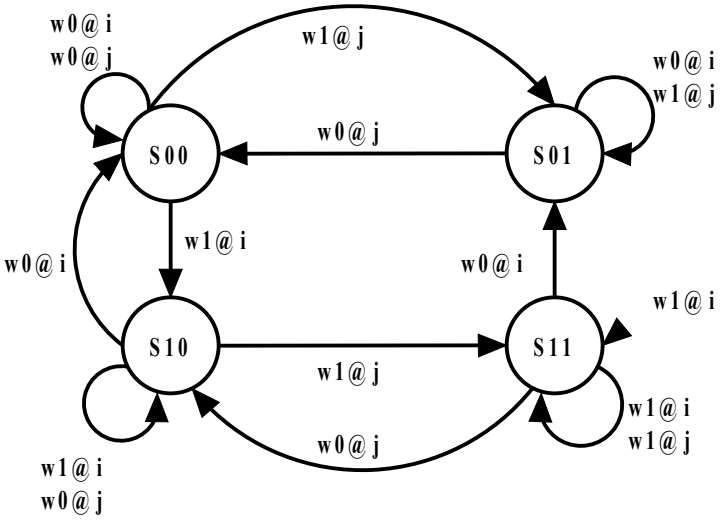


Figure 5. State diagram for two memory cells (i and j) under normal condition

Figure 6 shows the state machine for two cells i and j under rising inversion coupling fault $cfinv_{i,j}$. If we compare Figure 5 and Figure 6 we note that under normal condition if we write 1 to cell j (from state $S00$) we go to state $S01$, however, under rising $cfinv_{i,j}$ we go to state $S11$. This is explained as follows. When we write 1 to cell j in $S00$ it makes a 0 to 1 (rising) transition. As j is the coupling cell it complements the value of the coupled cell i (from 0 to 1). In Figure 6 the faulty transitions are indicated by thick arrows; $S00$ to $S11$ and $S10$ to $S01$ are two such transitions.

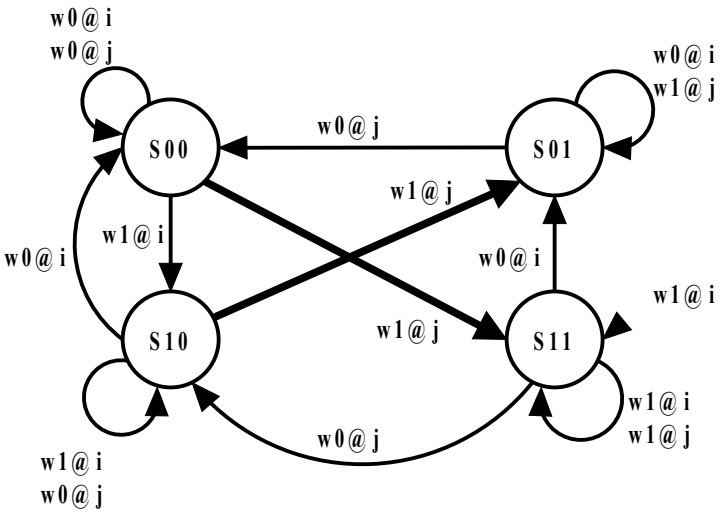


Figure 6. State diagram for memory cells (i, j) under $\uparrow$ inverse coupling fault $cfinv_{i,j}$

2.3.2 **Idempotent coupling faults**

In a 2-indempotent coupling fault $cfid_{i,j}$ say, involving cells i and j, a transition (0 to1 or 1 to 0) in memory cell j sets the value in memory cell i to be 0 or 1. The four possible 2- idempotent coupling faults involving cells i, j (denoted as $cfid_{i,j}$) are

- Rising-0: $\langle \uparrow | 0 \rangle$ (0 to1 change in cell j sets the content of cell i to be 0)
- Rising-1: $\langle \uparrow | 1 \rangle$ (0 to1 change in cell j sets the content of cell i to be 1)
- Falling-0: $\langle \downarrow | 0 \rangle$ (1 to 0 change in cell j sets the content of cell i to be 0)
- Falling-1: $\langle \downarrow | 1 \rangle$ (1 to 0 change in cell j sets the content of cell i to be 1)

Figure 7 shows the state machine for two cells i and j under rising-1

idempotent coupling fault $cfid_{i,j}$. If we compare Figure 5 and Figure 7 we note that under normal condition if we write 1 to cell j (from state $S00$) we go to state $S01$, however, under rising-1 $cfid_{i,j}$ we go to state $S11$. This situation is similar to $\uparrow$ inverse coupling fault (Figure 6). However, unlike $\uparrow$ inverse coupling fault (Figure 6), in rising-1 idempotent coupling fault we do not have a faulty transition from $S10$ to $S01$.

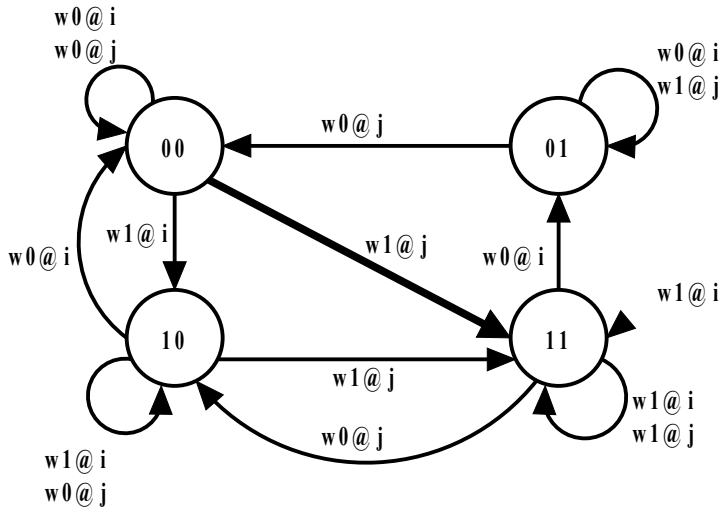


Figure 7. State diagram for memory cells (i, j)
under Rising-1 idempotent coupling fault

2.4 **Bridging fault**

A bridging fault is a short circuit between two or more cells. As in the case of coupling faults, to keep the number of faults within a practical number, it is assumed that only two cells can be involved in a bridging fault. There are two types of bridging faults

- AND bridging fault $ANDbf_{i,j}$ (involving cells i and j) which results in values in cells i and j to be logic AND of the values in these cells under normal condition. AND bridging fault is represented by $\langle v_i, v_j \mid v_i ANDv_j, v_i ANDv_j \rangle$ where the first two places represent the values in cells i and j under normal condition and the two values following "|"represent the values in cells i and j under AND bridging fault. $\langle 0,0|0,0\rangle, \langle 0,1|0,0\rangle, \langle 1,0|0,0\rangle, \langle 1,1|1,1\rangle$ are the four types of AND bridging faults possible.
- OR bridging fault $ORbf_{i,j}$ (involving cells i and j) which results in values in cells i and j to be logic OR of the values in these cells under normal condition. $\langle 0,0|0,0\rangle, \langle 0,1|1,1\rangle, \langle 1,0|1,1\rangle, \langle 1,1|1,1\rangle$ are the four types of OR bridging faults possible.

2.5 **Neighborhood pattern sensitive coupling faults**

One of the most important and different kind of fault in memory compared logic gate circuits is neighborhood pattern sensitive faults (NPSFs). As memory cells are very close to each other, the cells behave normally except for certain patterns in the neighborhood cells. For example, if a cell i has 0 and all the neighboring cells have 1, then the value of cell i may be pulled up to 1. It is obvious that given a cell there can be infinite number of neighborhood combinations. However for all practical cases there are two types of neighborhoods used in fault modeling for the cell under test.

- Type-1 neighborhood

  Figure 8 shows a type-1 neighborhood. The black colored cell is the one under test and the four cells around it (filled by small check boxes) are called neighborhood cells. Patterns in the neighborhood cells cause faults in the cell under test. The numbering of cells in type-1 neighborhood is shown in Figure 9.

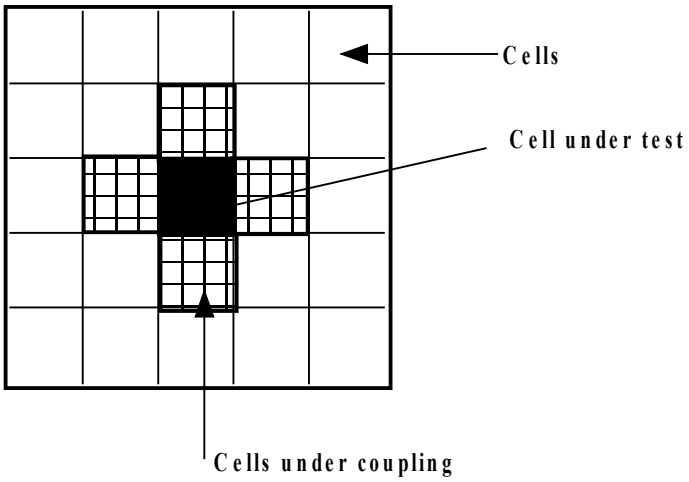


Figure 8. Type-1 neighborhood

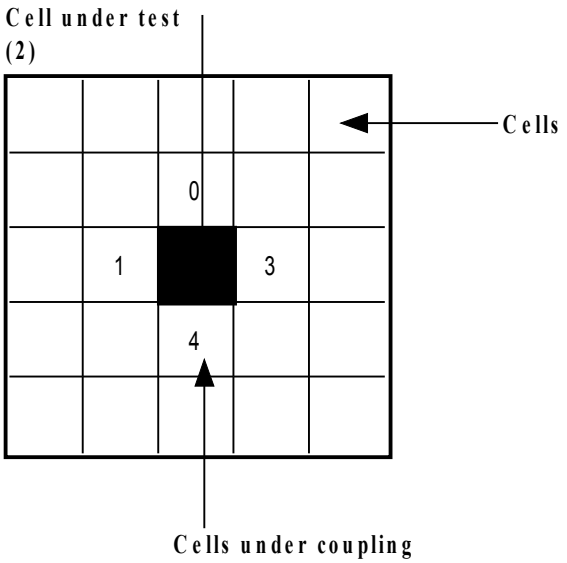


Figure 9. Numbering of cells in type-1 neighborhood

- Type-2 neighborhood

  A type-2 neighborhood has eight neighborhood cells corresponding to the cell under test. Type-2 neighborhood results in more complex fault model than a type-1 neighborhood. Figure 10 shows a type-2 neighborhood and the numbering of cells in type-2 neighborhood is shown in Figure 11.

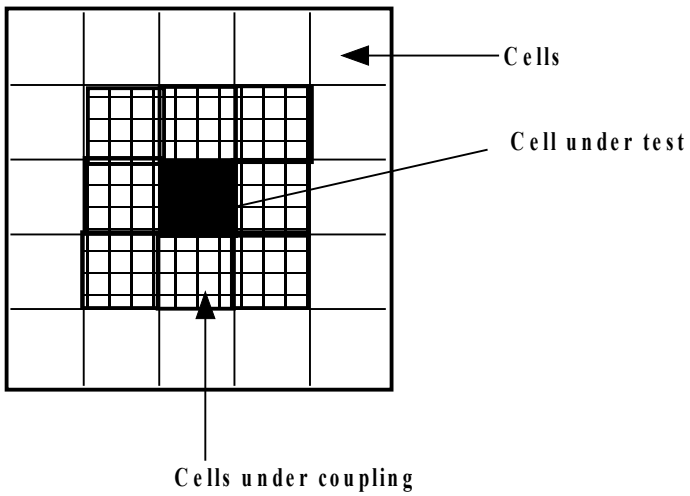


Figure 10. Type-2 neighborhood

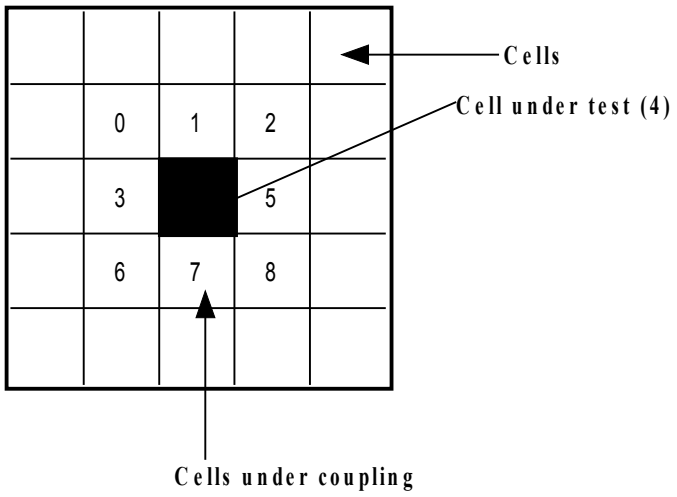


Figure 11. Numbering of cells in type-2 neighborhood

There are two types of NPSFs which are mainly used in fault modeling of memories.

- Active NPSF (ANPSF)

  The value in the cell under test changes due to a change in ONE cell of the neighborhood (type-1 or type-2 depending on the one being used); all other cells of the neighborhood make a pattern. An ANPSF is represented as "$^{vcut}\langle v_0, v_1, v_3, v_4 \mid fe \rangle$", where ^{vcut} is the value in the cell under test, v_0, v_1, v_3, v_4 represent the values in the neighboring cells (at cell no. 0,2,3,4 respectively) including the one which changes and fe represents fault effect in the cell under test. For example, $1\langle 0, \downarrow, 0, 0 \mid 0 \rangle$ represents the ANPSF were the cell under test initially has value of 1, the pattern made by neighboring cells is 0000 (values at cell no. 0,2,3,4 respectively) and fault effect at cell under test is 0 when a 1 to 0 transition in made in cell 2.

- Passive NPSF (PNPSF)

  PNPSF implies that a certain neighborhood pattern prevents the cell under test from changing its value. An PNPSF is represented as ^{vcut}

$\langle v_0, v_1, v_3, v_4 \mid fe \rangle$, where ^{vcut} is the value in the cell under test, v_0, v_1, v_3, v_4 represent the values in the neighboring cells and fe represents fault effect in the cell under test. There can be three types of fe PNPSF:

- $\uparrow \mid 0$: cell under test cannot be changed from 0 to 1 (initial value of cell under test is 0)
- $\downarrow \mid 1$: cell under test cannot be changed from 1 to 0 (initial value of cell under test is 1)
- $\updownarrow \mid x$: cell under test cannot be changed regardless of content.

In the above discussion of NPSFs we have taken only type-1 neighborhood, however, these models can be easily extended to type-2 neighborhood by increasing the number of cells in the NPSF representation accordingly.

2.6 **Address decoder faults**

Row and column decoder blocks of Figure 1 comprise the address decoder. As discussed before these blocks are composed of logic gates. However, their testing and fault models are different from that used to test circuits made of logic gates. From the context of memory testing four types of faults are considered in address decoder (for both reading and writing) [1]

- No cell is accessed for a certain address
- No address can access a certain cell
- With a particular address, multiple cells are simultaneously accessed
- A particular cell can be accessed with multiple addresses.

In the next section we will discuss test techniques for the fault models discussed.

3. **Testing of memory faults**

Before we discuss the test procedures for different fault models we introduce the concept of "March Test" which is used widely for memory testing [1,7]. March testing basically involves applying (writing and reading) patterns to each cell in memory before proceeding to the next cell and if a specific pattern is applied to one cell, then it must be applied to all cells. This is either done in increasing memory address order or decreasing order. Match test basically involves the following steps:

1. In increasing order of address of the memory cells, write 0s to the cells; illustrated in Figure 12.
2. In decreasing order of address of the memory cells, read the cells (expected value 0) and write 1 to the cells; illustrated in Figure 13.
3. In increasing order of address of the memory cells, read the cells (expected value 1) and write 0 to the cells; illustrated in Figure 13.
4. In decreasing order of address of the memory cells, read the cells (expected value 0); illustrated in Figure 13.

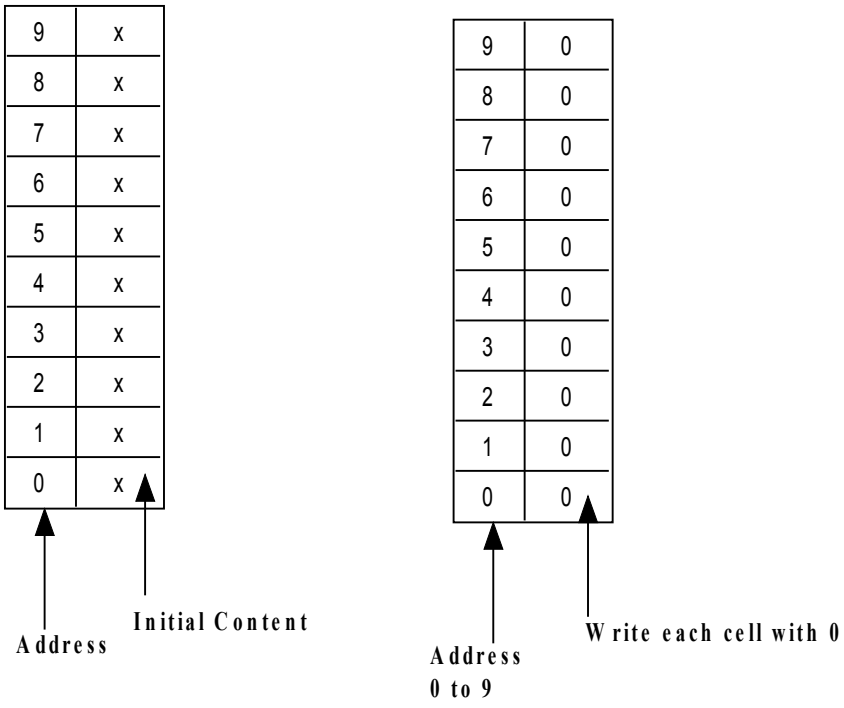


Figure 12. Step 1 of March test

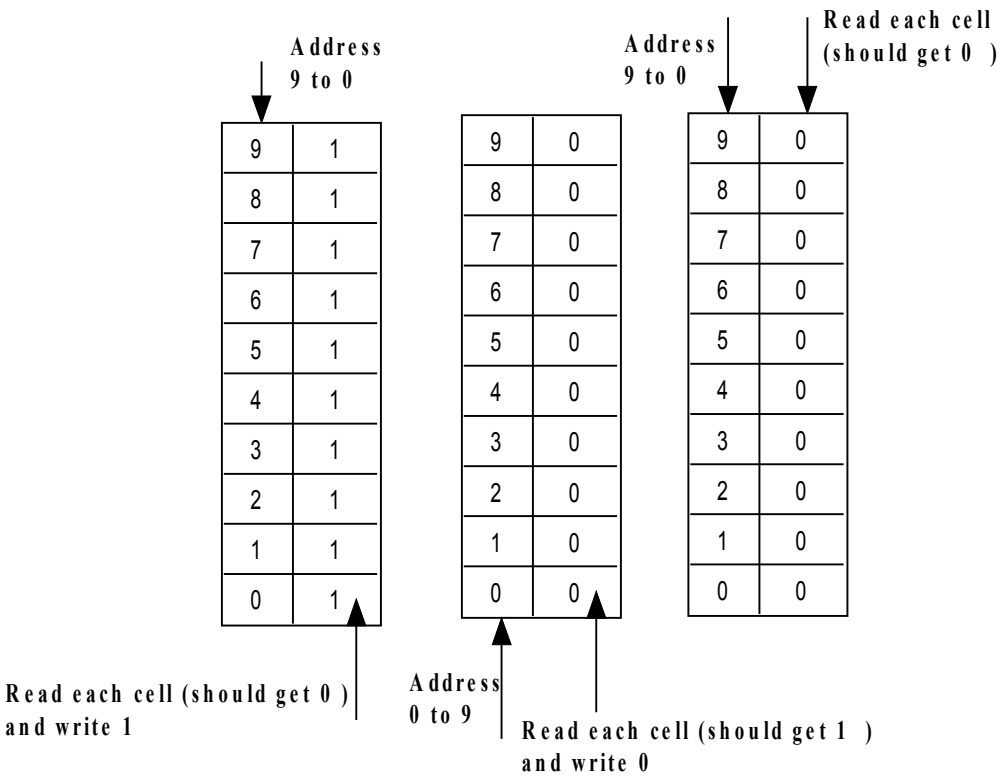


Figure 13. Step 2, Step 3 and Step 4 of March test

It may be noted that the steps of March test discussed above is only a guideline. One can start from highest address (9 in this case) and then go in decreasing order (in Step 1) to write 0s in the memory cells. Following that in Step-2, one needs to go from 0 to highest memory location. Similarly, we can start writing 1s in the cells in Step-1 and change the other steps accordingly.

For this section we will assume March test steps to be the ones given above. Now we will apply the basic concept of March test for the different fault models.

3.1 Stuck at fault model

March test discussed above obviously tests s-a-0 and s-a-1 faults in the cells because 0 and 1 in each cell is written and read back.

3.2 Transition fault

In March test during Step 1 all cells are written with 0 and in Step 2 all cells are written with 1s, thereby making a 0 to 1 transition in the cells. In Step 2 it is verified if cells have 0 in them and in Step 3 it is verified if cells have 1, thereby verifying 0 to 1 transition in the cells. So, Step 1 through Step 3 tests absence of $\langle \uparrow | 0 \rangle$ fault. In a similar manner, Step 3 through Step 5 tests absence of $\langle \downarrow | 0 \rangle$ fault.

3.3 Coupling Faults

March tests cannot detect all coupling faults. Let is consider three cells i, j, k such that address of $i <$ address of j and address of $j <$ address of k. Cell i is coupled with cell j and cell k by fault $\langle \uparrow | \updownarrow \rangle$; j and k are the coupling cell. In march test as we go either in ascending order or descending order of memory address of cells, both i and j are either visited before or after cell k. Let us consider the situation given in Figure 14. As Step-1 of March test all the cells i, j, k are written with 0. Following that in Step 2, all the cells (in order of) k, j, i are written with 1 (after successful reading of 0 from the cells). It may be noted that first cell k is written with 1; as cell i is coupled with cell k having fault $\langle \uparrow | \updownarrow \rangle$, the 0 to 1 transition in cell k inverts the content of cell i. Following that, cell j is written with 1; as cell i is also coupled with cell j having fault $\langle \uparrow | \updownarrow \rangle$, the 0 to 1 transition in cell j inverts the content of cell i again. Now when cell i is read, the value determined is 0

which means absence of two coupling faults (i) rising $cfinv_{i,j}$ and (ii) rising $cfinv_{i,k}$. In other words, "rising $cfinv_{i,k}$" masks "rising $cfinv_{i,j}$".

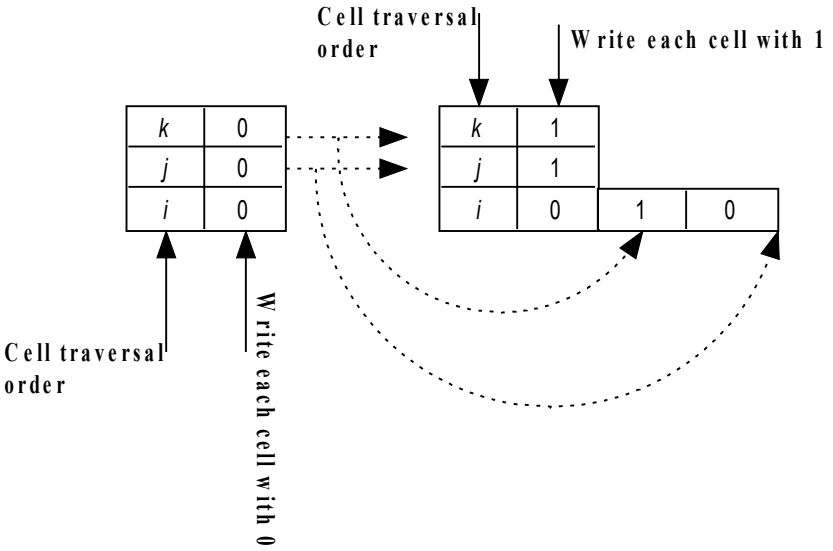


Figure 14. Example situation where March test fails to test coupling faults

So March tests cannot detect all coupling faults.

Test pattern sequence required to test different coupling faults are given below:

- Inverting rising coupling fault $\langle \uparrow \Updownarrow \rangle$ between cell i (*coupled cell*) and j (*coupling cell*): (i) Cell j is to be written with a 0 and read back, (ii) value at cell i is to be read and remembered, (iii) cell j is to be written with a 1 and read back, and (iv) value at cell i is to be read and checked that it is same as the one remembered (i.e., no inversion has happened).
- Inverting falling coupling fault $\langle \downarrow \Updownarrow \rangle$ between cell i and j: (i) Cell j is to be written with a 1 and read back, (ii) value at cell i is to be read and remembered, (iii) cell j is to be written with a 0 and read back, and (iv) value at cell i is to be read and checked that it is same as the one remembered (i.e., no inversion has happened).

- Idempotent Rising-0 coupling fault $\langle \uparrow | 0 \rangle$ between cell i and j: (i) Cell j is to be written with a 0 and read back, (ii) cell i is to be written with 1 and read back, (iii) cell j is to be written with a 1 and read back, and (iv) value at cell i is to be read and checked to be 1.
- Idempotent Rising-1 coupling fault $\langle \uparrow | 1 \rangle$ between cell i and j: (i) Cell j is to be written with a 0 and read back, (ii) cell i is to be written with 0 and read back, (iii) cell j is to be written with a 1 and read back, and (iv) value at cell i is to be read and checked to be 0.
- Idempotent Falling-0 coupling fault $\langle \downarrow | 0 \rangle$ between cell i and j: (i) Cell j is to be written with a 1 and read back, (ii) cell i is to be written with 1 and read back, (iii) cell j is to be written with a 0 and read back, and (iv) value at cell i is to be read and checked to be 1.
- Idempotent Falling-1 coupling fault $\langle \downarrow | 1 \rangle$ between cell i and j: (i) Cell j is to be written with a 1 and read back, (ii) cell i is to be written with 0 and read back, (iii) cell j is to be written with a 0 and read back, and (iv) value at cell i is to be read and checked to be 0.

The same test procedure is to be repeated for all combinations of coupled and coupling cells in the memory.

3.4 Bridging faults

Like coupling faults March tests cannot detect all bridging faults. As discussed in Section 2.4, $\langle 0,0|0,0\rangle, \langle 0,1|0,0\rangle, \langle 1,0|0,0\rangle, \langle 1,1|1,1\rangle$ are the four types of AND bridging faults possible. This implies that cells i, j which are involved in bridging faults must have the four combinations of inputs 00,01,10 and 11. From Figure 12 and Figure 13, it may be noted that no cell pairs have all the four combinations 00,01,10 and 11. So to test bridging faults the following test pattern sequences are required.

- AND bridging fault $ANDbf_{i,j}$ (involving cells i and j): (i) write 0 in cell i and 0 in cell j and read back the values (which must remain same), (ii) write 0 in cell i and 1 in cell j and read back the values, (iii) write 1 in cell i and 0 in cell j and read back the values, and (iv) write 1 in cell i and 1 in cell j and read back the values.

It may be noted that the above four test pattern sequence are enough to test OR bridging fault also because we write all possible combinations in the two cells (involved in fault) and read back to check if they retain their values.

3.5 Neighborhood pattern sensitive coupling faults (NPSF)

Similar to the case of bridging and coupling faults, March tests cannot detect NPSFs. For each NPSF we need a particular sequence (according to the one given in the fault) to be given in the neighboring cells of the cell under test. Also the cell under test is to be written in a way such that fault is sensitized. Finally, we need to make the transition(s) in the pattern(s) in the neighboring cells and verify the value in the cell under test. Now we give example of testing one active NPSF and one passive NPSF (using neighborhood type-1).

- **Example of testing active NPSF** $1\langle 0,\downarrow,0,0\,|\,0\rangle$

  (i) write 0,1,0,0 in cell no. 0,2,3,4 respectively, of the type-1 neighborhood (of the cell under test) and read back the values, (ii) write 1 in cell under test and read back the value, (iii) write 0 in cell 2 and (iv) read the value in cell under test (which is to be 1).

- **Example of testing passive NPSF** $\langle 0,0,0,0\,|\,\langle\uparrow|\,0\rangle\rangle$

  (i) write 0,0,0,0 in cell no. 0,2,3,4 respectively, of the type-1 neighborhood (of the cell under test) and read back the values, (ii) write 0 in cell under test and read back the value, (iii) write 1 in cell under test and (iv) read the value in cell under test (which is to be 1).

3.6 Address decoder faults

A little variation of March test (given in Figure 12 and Figure 13) can test all four address decoder faults given in Section 2.6. We will only give the test sequence (of modified March test) and proof idea that it tests all four address decoder faults. The formal proof is given in [1].

The test pattern sequence for modified March test is given below.

- In increasing order of address of the memory cells, read the value of the memory cells and write complement value in the cell. If 1 is read at cell 0, value of 1 is written to cell 0; following that same procedure is followed for cell 2 and so on for entire memory.
- In decreasing order of address of the memory cells, read the cells (match with expected value) and write complement value in the cell.

The basic principle is that as the memory writing and examination operation moves through memory, any address decoder fault that causes unexpected accesses of memory locations will cause those locations to be written to an unexpected value. As the test proceeds, it will discover those locations and report a fault.

4. **Basics of memory BIST**

From the discussion in last section it may be noted that for March test an address generator (increasing and decreasing order) and a data reader cum writer is required. So BIST for March test will be simply an LFSR and a data reader cum writer. As in the case of logic BIST, the LFSR should have primitive polynomial (so that it generates all numbers from 1 to 2^n), and along with this the LFSR for memory BIST should the following features

- Be able to generate all the 0 pattern to access the last memory location
- Be able to generate forward and reverse order patterns i.e., if 1-0-2-3 be the sequence of the LFSR (when initialized with 1) then there should be a mode to generate the sequence 1-3-2-0.

It may be noted that March test can be modified by replacing "sequential read/write" with "arbitrary order read/write, but covering all cells" without loss in test capability. In other words, in Figure 12 and Figure 13, we illustrated cell traversal from 0 to 9 and then from 9 to 0. However, the test capability will not change if sequence of cell traversal is any other sequence, for example, 1-0-2-5-7-3-4-6-9-8 while moving in ascending order and 1-8-9-6-4-3-7-5-2-0 in reverse order.

In this section we will discuss LFSR design that satisfies the above mentioned two points which can be used for test pattern sequence generation in March test. BIST circuitry for testing faults that involve participation of more than one cell like coupling faults, NPSFs etc. are more complex and will not be covered in this lecture; for details the student is referred to [8,9].

We will modify the standard LFSR discussed in Figure 4 of Lecture 1 of this module (Module XIII) so that it satisfies the above mentioned two points for memory BIST; Figure 15 illustrates the modified LFSR, where the positions of the flip-flops are in reverse order, i.e., X_0 (LFSR of Figure 4 of Lecture 1) is marked as X_2 and X_2 (LFSR of Figure 4 of Lecture 1) is marked as X_0. Also, circuitry for feedback from last flip-flop to first flip-flop now involves an extra XOR gate and a NOR gate. It may be noted from the sequence of outputs of Figure 15 that the all-0 pattern is generated from the pattern

$X_0X_1X_2 = 001$ because of the extra NOR+XOR gates. If the circuitry for feedback from last flip-flop to first flip-flop comprises only XOR gates (as in LFSR of Figure 4 of Lecture 1), then the all-0 pattern cannot be generated from any pattern in $X_0X_1X_2$ (except $X_0X_1X_2 = 000$). If we start from the all-0 pattern in the LFSR of Figure 4 of Lecture 1 it does not change its output.

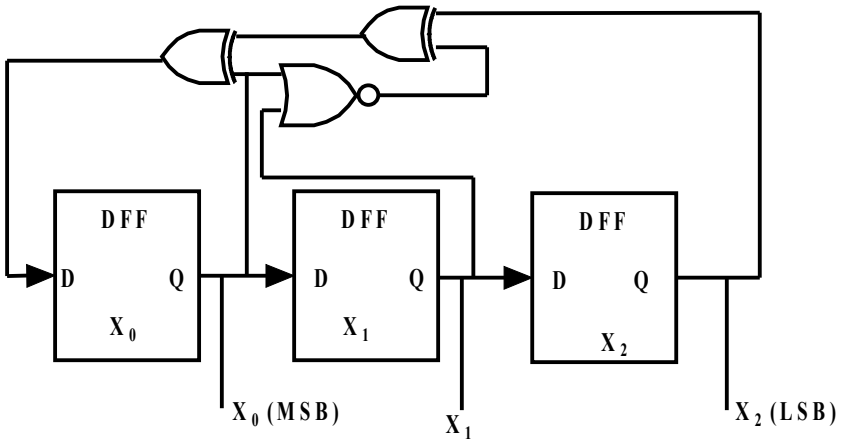


Figure 15 LFSR to generate up address sequence for memory BIST

The output sequence generated by the LFSR of Figure 15 is given below.

$$\begin{bmatrix} X_0(MSB) \\ X_1 \\ X_2(LSB) \end{bmatrix} ::: \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

It may be noted that the LFSR generates all patterns in the range $1 \text{ to } (2^3 - 1)$ along with the 000 pattern. The seed of the LFSR is 001. The pattern sequence in decimal is 1-0-4-6-7-3-5-2.

As already discussed, for March test the LFSR (Figure 15) should also have a counterpart that can generates a reverse pattern i.e., the reverse LFSR should generate reverse of "1-0-4-6-7-3-5-2", which is 1-2-5-3-7-6-4-0 (seed is assumed to be same for both the LFSR and its reverse counterpart). Broadly speaking, the reverse LFSR should shift the data in the reverse direction. So to achieve this reverse shift of data, the D and Q terminals are to be swapped in the (forward) LFSR and numbering of the flip-flops is to be made in

reverse order. The feedback from last flip-flop to the first one will also have the XOR-NOR gate circuitry (as in the case of forward LFSR) to generate the all-0 sequence. Figure 16 shows the reverse LFSR (for the forward one shown in Figure 15).

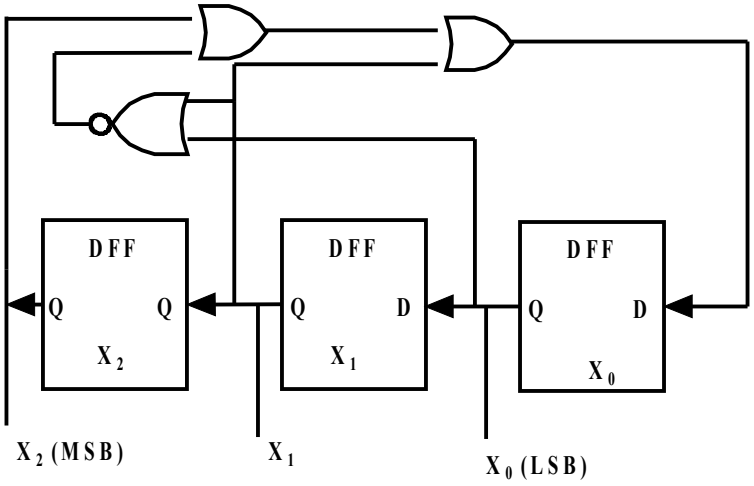


Figure 16. Reverse LFSR of the (forward) LFSR in Figure 15

The output sequence generated by the LFSR of Figure 16 is given below.

$$\begin{bmatrix} X_0(LSB) \\ X_1 \\ X_2(MSB) \end{bmatrix} ::: \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

A broad architecture for memory BIST using LFSR is shown in Figure 17. The various blocks are explained as follows

- LFSR: The LFSR block comprises both forward and reverse LFSRs. This block is used to generate address of the memory cells where read/write is to be done

- Data: The data block to writes 0 and 1 in the cells as per steps of March test.

- Equality comparator: In general, most of the ICs have their memory divided into have multiple arrays (or memory blocks). In such a case we can test two or more arrays (in this case 2) simultaneously, by applying the same addresses and data (being written, as per March test) to all the

arrays. If same address and data is applied, outputs of all the memory blocks are to be same. The mutual comparator ensures that outputs of all blocks are equal and asserts the error signal when output of some block disagrees with that of others. The comparator eliminates the need to generate the good machine response, response compaction etc.

- Controller: It basically coordinates the BIST procedure. It selects forward LFSR and backward LFSR alternatively to generate address for memory cells. It also enables the data writer block to write 0 and 1 as per steps of March test. Further, it also enables the equality comparator to check if outputs of all memory blocks are equal.

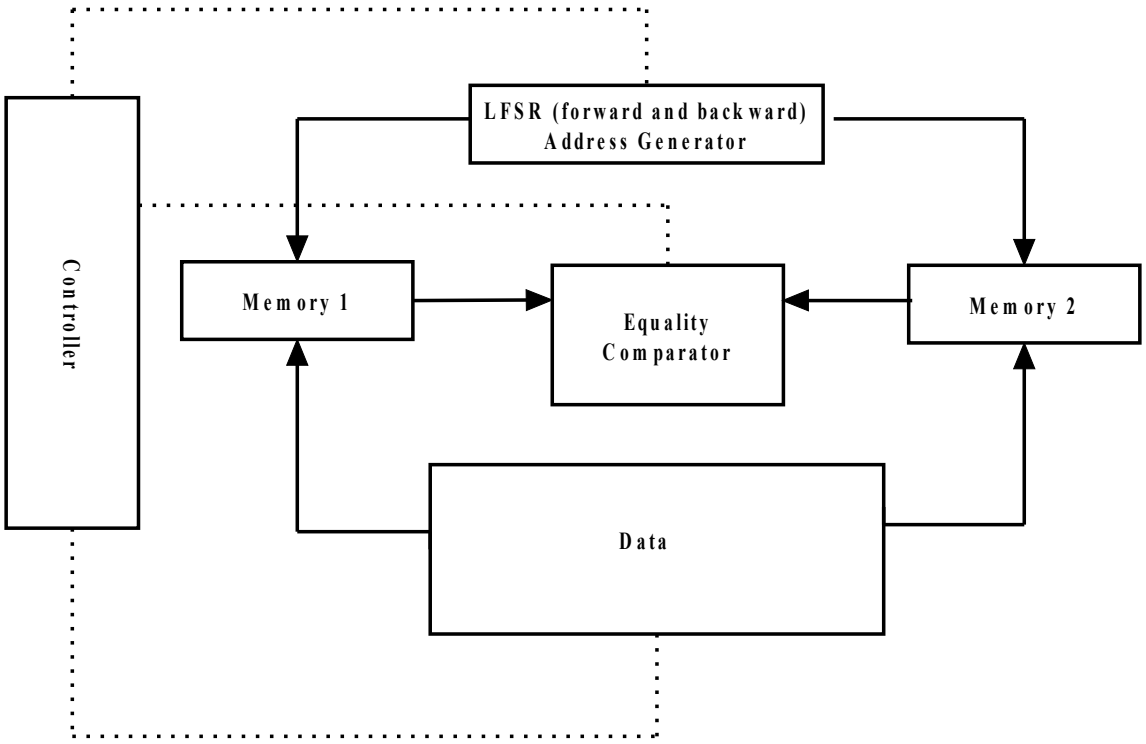


Figure 17. Memory BIST using LFSR

5. Conclusions

All the lectures covered in the Digital Testing module except, the present one were focused on testing of digital circuits comprising Boolean gates and flip-flops. However, memory is a very vital element for digital circuits. The structure of memory blocks are basically different compared to logic blocks. In this lecture we have covered simple logical fault models applicable to digital memories and the corresponding test procedures. Also we have given a brief idea about BIST of memories for March test.

Most of the discussions in this lecture were focused at logical fault models in memory which are independent of the fabrication technology, their physical layout, electrical parameters etc. The advantage of logical models is that they have enough detail to adequately model a good number of faults like stuck-at, transition, coupling, etc. and at the same time simple to handle. However, in memory blocks fabricated with latest technologies we need to test for more advanced type of faults which are based on timing parameters, geometry of the layout etc. Timing tests are required to validate the read/write speed of the memory block as per specifications. The geometrical fault model of a memory requires complete knowledge of the chip layout. This allows inductive fault analysis [10] to be used during initial memory chip production when we are trying to redesign the memory layout to increase the memory chip yield. The details of memory testing for advanced fault models can be found in [1,6,10].

References

1. A. J. van de Goor, Testing Semiconductor Memories: Theory and Practice. Chichester, UK: John Wiley & Sons, Inc., 1991.
2. P. Mazumder and K. Chakraborty, Testing and Testable Design of High-Density Random-Access Memories. Boston: Kluwer Academic Publishers, 1996.
3. Ashok K. Sharma, "Advanced Semiconductor Memories: Architectures, Designs, and Applications", Wiley-IEEE Press, 2002.
4. R. Nair, S. M. Thatte, and J. A. Abraham, "Efficient Algorithms for Testing Semiconductor Random-Access Memories," IEEE Trans. on Computers, vol. C-28, no. 3, pp. 572–576, Mar. 1978.
5. S. M. Thatte and J. A. Abraham, "Testing of Semiconductor Random Access Memories," in Proc. of the International Fault-Tolerant Computing Symp., June 1977, pp. 81–87.
6. Ashok K. Sharma, "Semiconductor Memories: Technology, Testing, and Reliability", Wiley-IEEE Press, 2002
7. D. S. Suk and S. M. Reddy, "A March Test for Functional Faults in Semiconductor Random-Access Memories," IEEE Trans. on Computers, vol. C-30, no. 12, pp. 982– 985, Dec. 1981.
8. K. Kinoshita and K. K. Saluja, "Built-in Testing of Memory Using On-Chip Compact Testing Scheme," in Proc. of the International Test Conf., Oct. 1984, pp. 271–281.
9. K. Kinoshita and K. K. Saluja, "Built-in Testing of Memory Using an On-Chip Compact Testing Scheme," IEEE Trans. on Computers, vol. C-35, no. 10, pp. 862–870, Oct. 1986.
10. P. Mazumder and K. Chakraborty, Testing and Testable Design of High-Density Random-Access Memories. Boston: Kluwer Academic Publishers, 1996.

Questions and Answers

1. What is the total number of all possible (i) passive NPSFs and (ii) active NPSF in type-1 and type-2 neighborhood?

Answer:

- Passive NPSF:

  As we know there are three types of passive NPSF as: $\uparrow|0$, $\downarrow|1$ and $\updownarrow|x$. In case of type-1 neighborhood, as there are 4 neighborhood cells there can be 2^4 pattern combinations. So number of passive NPSFs can be $3x\,2^4$. In case of type-2 neighborhood, there are 7 neighborhood cells thereby making the number of passive NPSFs to be $3x\,2^7$.

- Active NPSF

  In active NPSF ($^{vcut}\langle v_0, v_1, v_3, v_4 \mid fe\rangle$) there are two types of faults as: $vcut=1,\ fe=0$ and $vcut=0,\ fe=1$. Each neighborhood cell can have four combinations: $0,1,\uparrow,\downarrow$. In type-1 neighborhood there are 4 neighborhood cells and each can have 4 combinations, so there can be 4^4 pattern combinations. So number of passive NPSFs can be $2x\,4^4$.

  In case of type-2 neighborhood, there are 7 neighborhood cells thereby making the number of active NPSFs to be $2x\,4^7$.

2. From answer 1 it is clear that testing type-1 neighborhood is more complex than type-2 neighborhood. What assumption regarding fault model is made when type-1 neighborhood is adopted instead of type-2 neighborhood.

Answer:

Type-1 neighborhood assumes either that diagonal cell couplings are not prominent or that if there is a diagonal cell coupling, then the fault will also cause a vertical or horizontal cell coupling. Type-2 neighborhood is needed when diagonal couplings are significant, and do not necessarily cause a vertical/horizontal coupling.